

■ Discover Sénégal

Guide d'Améliorations — Projet de Mémoire M2 SIR

Master 2 Systèmes d'Information et Réseaux • 2025 / 2026

Ce document récapitule l'ensemble des améliorations proposées pour le projet **Discover Sénégal** afin d'en faire un mémoire de niveau M2 remarquable. Il couvre l'architecture distribuée, l'observabilité, la résilience, le déploiement et les fonctionnalités applicatives, avec pour chaque axe le contexte, les outils recommandés et les actions concrètes à mener.

#	Thème	Priorité
1	Observabilité & Monitoring distribué	■ Haute
2	Résilience & Tolérance aux pannes	■ Haute
3	Conteneurisation avancée (Docker → K8s)	■ Moyenne
4	Fonctionnalités applicatives	■ Haute
5	Gestion des photos (Cloudinary/S3)	■ Moyenne
6	Itinéraire touristique personnalisé	■ Bonus impressionnant

AXE 1 — Observabilité & Monitoring Distribué

Pourquoi c'est indispensable ?

Dans une architecture microservices, lorsqu'une requête échoue, il est impossible de savoir dans quel service le problème se situe sans outillage dédié. L'observabilité repose sur trois piliers complémentaires :

- **Logs** : ce qui s'est passé dans chaque service
- **Métriques** : comment le système se comporte (CPU, temps de réponse, nb requêtes)
- **Traces distribuées** : le chemin complet d'une requête à travers tous les microservices

Stack technique recommandée

Outil	Rôle	Intégration Spring Boot
Zipkin	Tracing distribué — visualise le chemin d'une requête entre services	spring-cloud-starter-zipkin + Sleuth
Spring Actuator + Micrometer	Expose les métriques de chaque service via /actuator/metrics	spring-boot-starter-actuator micrometer-registry-prometheus
Prometheus	Collecte et stocke les métriques de tous les services	Configuration prometheus.yml
Grafana	Dashboard visuel temps réel (graphes, alertes, KPIs)	Connexion à Prometheus

Dépendances Maven à ajouter (pom.xml)

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-sleuth</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-sleuth-zipkin</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Configuration application.yml

```
spring:
  zipkin:
    base-url: http://zipkin:9411
  sleuth:
    sampler:
```

```
    probability: 1.0
management:
  endpoints:
    web:
      exposure:
        include: '*'
```

Ce que ça apporte au mémoire

Avec Grafana, tu peux présenter au jury des **dashboards réels** montrant les temps de réponse de chaque service, le nombre de requêtes par minute, et identifier les goulots d'étranglement. Avec Zipkin, tu montres visuellement le chemin d'une requête : Gateway → Auth → Tourisme → Recommendation, avec le temps passé dans chaque service. C'est un argument académique et visuel très fort.

AXE 2 — Résilience & Tolérance aux Pannes

Le problème sans résilience

Sans résilience, si le service **notification** tombe, et que **reservation** l'appelle via Feign, toute la chaîne plante en cascade. C'est la **défaillance en cascade** — le scénario catastrophe des architectures microservices. Resilience4j permet d'éviter ça.

Les 4 patterns Resilience4j

Pattern	Description	Cas d'usage dans Discover Sénégal
Circuit Breaker	Après X échecs, le circuit s'ouvre. On retourne un fallback. Se referme automatiquement.	Si recommandation-service est KO, tourisme-service retourne une liste par défaut.
Retry	En cas d'échec, on réessaie N fois avec délai exponentiel.	Retry sur l'appel au service notification en cas de timeout.
Rate Limiter	Limite le nb d'appels/seconde pour protéger un service surchargé.	Protéger l'API publique de recherche de lieux contre les abus.
Bulkhead	Isole les ressources pour qu'un service surchargé n'affecte pas les autres.	Isoler le service réservation du reste de l'application.

Dépendance Maven

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot2</artifactId>
  <version>1.7.1</version>
</dependency>
```

Exemple Circuit Breaker sur le service Tourisme

```
@CircuitBreaker(name = "recommendationService", fallbackMethod =
"defaultRecommendations")
public List<Lieu> getRecommendations(Long userId) {
    return recommendationClient.getForUser(userId);
}

public List<Lieu> defaultRecommendations(Long userId, Exception e) {
    // Retourne les lieux les mieux notés par défaut
    return lieuRepository.findTop10ByOrderByNoteDesc();
}
```

Configuration resilience4j (application.yml)

```
resilience4j:
  circuitbreaker:
    instances:
      recommendationService:
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 10000
        permittedNumberOfCallsInHalfOpenState: 3
```

AXE 3 — Conteneurisation Avancée : Docker Compose → Kubernetes

Tu as déjà Docker Compose — c'est très bien !

Voici ce que Kubernetes apporterait en plus pour ton mémoire :

Capacité	Docker Compose	Kubernetes
Redémarrage auto si crash	Non	Oui (always restart)
Auto-scaling	Manuel	Automatique selon la charge
Multi-nœuds / Haute dispo	Non (1 seul serveur)	Oui
Rolling updates	Non	Oui, sans downtime
Health checks avancés	Basiques	Liveness + Readiness probes
Load balancing interne	Basique	Service K8s natif

Stratégie recommandée : Tu n'as pas besoin de tout migrer vers Kubernetes. Pour le mémoire, il suffit de montrer un fichier **deployment.yaml** + **service.yaml** pour 2 ou 3 microservices, et de l'exécuter sur **Minikube** en local. Cela démontre que tu maîtrises le passage à l'échelle et que ton architecture est prête pour la production.

Exemple deployment.yaml (service tourisme)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tourisme-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: tourisme-service
  template:
    spec:
      containers:
        - name: tourisme-service
          image: discover-senegal/tourisme:latest
          ports:
            - containerPort: 8082
          livenessProbe:
            httpGet:
              path: /actuator/health
              port: 8082
```

État actuel vs. Améliorations proposées

Ce qui est déjà fait ✓

- ✓ Enregistrement des hôtels, restaurants, plages, lieux religieux et culturels
- ✓ Géolocalisation (carte intégrée)
- ✓ Système d'avis et de notation par étoiles (sans calcul de moyenne)
- ✓ Tableau de bord admin (ajout / modification / suppression des entités)
- ✓ Réservation (statique pour l'instant)

Ce qui est à compléter ou améliorer →

- Calcul automatique de la moyenne des notes par lieu
- Afficher les lieux sur la carte (markers pour hôtels, restaurants, plages...)
- Filtres de recherche : par ville, catégorie, fourchette de prix, note minimale
- Réservation avec statuts : EN_ATTENTE → CONFIRMÉE → ANNULÉE
- Notification Flutter (push) à chaque changement de statut de réservation
- Galerie photos par lieu (stockage Cloudinary ou AWS S3)

Fonctionnalité bonus — Itinéraire touristique personnalisé

L'utilisateur renseigne ses préférences (plage, culture, gastronomie, religion) et le nombre de jours. L'application génère un **circuit sur mesure** au Sénégal (ex : Jour 1 Dakar → Jour 2 Saint-Louis → Jour 3 Ziguinchor). C'est là que le service **recommandation** prend tout son sens académique. Algorithme suggéré : filtrage collaboratif ou scoring par catégorie pondérée.

Détail : Réservation avec statuts (Spring Boot)

```
public enum StatutReservation {
    EN_ATTENTE, CONFIRMEE, ANNULEE, TERMINEE
}

// Dans ReservationService.java
public Reservation confirmer(Long id) {
    Reservation r = reservationRepo.findById(id).orElseThrow();
    r.setStatut(StatutReservation.CONFIRMEE);
    notificationClient.envoyer(r.getUserId(),
        "Votre réservation a été confirmée !");
    return reservationRepo.save(r);
}
```

AXE 5 — Gestion des Photos (Cloudinary / AWS S3)

Pourquoi ne pas stocker les images en base ?

Stocker des images directement en base de données (BLOB) est une très mauvaise pratique en production : cela alourdit massivement la BDD et ralentit toutes les requêtes. La bonne approche est de stocker les images sur un service dédié (Cloudinary ou AWS S3) et de ne garder en base que l'**URL** de l'image.

Critère	Cloudinary	AWS S3
Complexité	Très simple — SDK direct	Moyenne — config IAM
Gratuit	Oui (25 Go gratuits)	Limité (12 mois free tier)
Transformations image	Oui (resize, crop, filtre)	Non (natif)
CDN intégré	Oui	Oui (CloudFront)
Recommandé pour M2	✓ Oui — plus simple	Possible mais plus complexe

Intégration Cloudinary — Spring Boot

```
# application.yml
cloudinary:
  cloud-name: ton-cloud-name
  api-key: ta-cle-api
  api-secret: ton-secret

// CloudinaryService.java
@Service
public class CloudinaryService {
    @Autowired private Cloudinary cloudinary;

    public String uploadImage(MultipartFile file) throws IOException {
        Map result = cloudinary.uploader()
            .upload(file.getBytes(), ObjectUtils.emptyMap());
        return result.get("secure_url").toString();
    }
}
```

Côté Flutter — Upload de photo

```
// Sélection image avec image_picker
final picker = ImagePicker();
final file = await picker.pickImage(source: ImageSource.gallery);

// Upload via multipart request vers ton backend Spring Boot
```



```
var request = http.MultipartRequest('POST',  
    Uri.parse('http://gateway/api/lieux/\$lieuId/photos'));  
request.files.add(await http.MultipartFile.fromPath('file', file!.path));  
var response = await request.send();
```

Feuille de Route — Plan d'Action Recommandé

Phase	Actions	Impact mémoire
Phase 1 (Fondations)	• Calcul moyenne des notes • Filtres recherche (ville, catégorie) • Markers sur la carte	Fonctionnalité complète et cohérente
Phase 2 (Architecture)	• Resilience4j (Circuit Breaker) • Zipkin + Sleuth (tracing) • Actuator + Prometheus	Démonstration de maturité M2 SIR
Phase 3 (Production)	• Cloudinary pour les photos • Statuts de réservation + notifications • Grafana dashboard	Projet prêt pour le déploiement réel
Phase 4 (Excellence)	• Itinéraire personnalisé • Kubernetes (Minikube) • Tests de charge JMeter	Jury surpris, note maximale

Conseil final : Pour le mémoire académique, assure-toi d'avoir une problématique claire du type : *"Comment concevoir une architecture microservices résiliente, observable et scalable pour une application touristique dans un contexte africain ?"*. Chaque amélioration technique doit être justifiée dans le texte du mémoire par rapport à cette problématique. Les dashboards Grafana, les traces Zipkin et les tests de résilience seront tes preuves expérimentales.